

BOOST – BOUNDED fORward Symbolic execuTion verification

Olaf Boekholt
Job Vonk*

Abstract

We present BOOST, a bounded forward symbolic execution verifier for the Guarded Command Language (GCL). BOOST aims to efficiently verify program properties by combining symbolic execution with practical optimisations such as formula simplification and path pruning. Our implementation supports exception handling and leverages SAT solving to reason about symbolic constraints over bounded program paths.

We evaluate BOOST on several benchmark programs that feature branching, loops, and array operations. The experiments show that BOOST achieves completeness for all invalid cases while maintaining practical runtime performance. The simplifier substantially reduces solver load by collapsing complex symbolic expressions, whereas pruning heuristics successfully eliminate infeasible execution paths to mitigate path explosion. Together, these techniques demonstrate that BOOST provides a sound and efficient framework for bounded program verification, balancing correctness, scalability, and usability.

Keywords

Bounded Forwards Symbolic Execution, Program Verification

1 Introduction

There are different tools programmers can use to reduce how many bugs are created during development of a new software system. One approach is to use testing: the programmers write pieces of code that assert that another piece of code produces the expected results given some pre-defined inputs. While this approach works, and is used often in industry, there are some drawbacks: programmers need to write the tests, which can be quite cumbersome. Instead, it would be more efficient if the programmer could specify what the properties are that should hold, and that a separate tool verifies whether these properties hold. One class of such tools is tools based on bounded symbolic execution. These tools run the program on symbolic variables and uses SAT solvers to verify whether properties hold at certain points in the program. For example, the following program has a bug where a division by error is thrown:

```
reciprocals n = map (1 /) [0..n]
```

Forwards symbolic verification engines can prevent this bug, by warning the programmer about a precondition violation for division. The division operator can add the additional constraint that the nominator cannot be 0, which is then checked in every invocation of the operator. If evidence cannot be established – like in the example – then the program is rejected. In this paper, we will look into BOOST: a forwards bounded symbolic verification engine. BOOST verifies properties in the GCL programming language with support for exception handling as well. It implements some techniques like path pruning and formula simplification to efficiently execute

programs. In particular, we will discuss BOOST’s implementation, optimisations and evaluate how it performs on various programs.

2 Related work

There have been numerous works published relating to symbolic execution based verification. Baldoni et al. provide a good background on symbolic execution techniques [1]. They describe how a satisfiability modulo theories (SMT) solver can be used to determine feasible paths and generate concrete inputs that satisfy them.

Despite its effectiveness in systematically exploring program paths and generating high-coverage test inputs, symbolic execution faces several key challenges that limit its scalability in practice [3]. The path explosion problem arises because the number of feasible paths grows exponentially with program branches and loops, making full exploration infeasible. Constraint solving is another major bottleneck, as SMT solvers struggle with large or complex path conditions. Furthermore, memory modeling requires a trade-off between precision and performance; simplified models may miss bugs, while precise bit-level reasoning slows execution. Finally, symbolic execution struggles with concurrency and environment interactions, where thread interleavings and external calls increase nondeterminism and require approximations or concrete execution fallbacks.

To tackle the constraint solving cost and path explosion Kapus, Busse, and Cadar try to enhance scalability by aggressively prioritising execution paths that are already known to be feasible [4].

Schwartz, Avgerinos, and Brumley look at forward symbolic execution and dynamic taint analysis. They highlight challenges, different techniques, and tradeoffs when using these techniques in a security setting [6].

We have identified two main approaches to improve the performance of symbolic execution in the existing literature; state merging and solver result caching.

2.1 State merging

It may be profitable to apply state merging whenever two symbolic states are similar [1]. However, this requires a fine balance, as spending time calculating the equivalence of states could outweigh the performance gained by preventing a state explosion. Kuznetsov et al. [5] analysed the trade-offs of merging execution states in symbolic execution. While merging can reduce the number of explored paths, it increases the complexity of the symbolic expressions and solver queries, often degrading performance. Their work introduced query count estimation and dynamic state merging to decide when merging is beneficial, achieving large speedups in practical situation.

Trabish et al. propose a specialised solving procedure by dynamically partitioning the symbolic states into merging groups. Similar

*Both authors contributed equally to this research.

structures detected in their path constraints allow efficient encoding of the merged path constraint and memory using quantifiers [8].

2.2 Solver result caching

Another way to improve the performance of symbolic execution can be to cache intermediate results of the solver. Cadar, Dunbar, and Engler introduced KLEE, a symbolic execution tool that utilizes a counter-example cache that maps sets of constraints to counter-examples to increase performance [2].

Constructing this cache could be tricky in some cases, so Shuai et al. propose a partial solution-based caching method to improve its effectiveness [7]. They use partial solutions inside the constraint solving to generate more cache entries.

3 Method

The full repository implementing the verifier can be found on Github¹.

3.1 Computation trees

We implemented our verification engine for a restricted subset of the guarded control language (GCL). Our subset includes:

- Variables, integer and boolean operators;
- Branching control flow, in the form of if-else statements;
- Iterative control flow, in the form of while statements;
- Exception throwing and handling.

For parsing the GCL syntax we used an existing library², whose abstract syntax trees (ASTs) we use as input to our verification engine. However, we need to transform the ASTs into a representation that allows us to reason more easily about the program's control flow. Therefore, we define a computation tree as described in Listing 1.

```
data Tree
= Empty
| Next Stmt Tree
| Branch Expr Tree Tree
| TWhile Expr Stmt Tree
```

Listing 1: The definition of computation trees that are built from ASTs.

The Next constructor represents composition of statements. We execute the statement stored in the constructor and then continue with the subtree. The Branch constructor handles conditional logic in the program. The expression is evaluated and depending on its result, either the first or second subtree is evaluated. The TWhile node is a special case that allows lazy unfolding limited by the supplied maximum program length.

Then we define a function that converts an AST to a computation tree. As a helper function we define a combination function on computation trees: $t1 \triangleleft t2$ replaces every Empty in $t1$ with $t2$. Together with the identity element Empty, this forms a monoid on Trees. With this function, we can define the tree building function

recursively over the AST. We highlight the cases for if-statements and sequencing in Listing 2.

```
buildTree = \case
Seq s1 s2 -> (<>) <$> buildTree s1 <*> buildTree s2
IfThenElse g t e -> Branch False
    <$> annotatePredicate g
    <*> buildTree t
    <*> buildTree e
```

Listing 2: Snippet of code for the buildTree function which converts a GCL AST into a computation tree.

3.1.1 Loop unfolding. Because the computation tree is a tree, it does not allow cycles in any path. So, in order to verify while loops – which are typically represented by cycles in control flow graphs – there is a single node in the computation tree that represents while loops. The unfolding of the while loop happens in the verifier, where the while node is rewritten into a branch node. Listing 3 shows a simplified version of how this unrolling happens.

```
verify t = asks pathLength >= \case
-- When the program length limit is reached, stop
0 -> stop
_ -> case t of
-- Unroll one iteration of the while loop
TWhile g b t -> verify $
    Branch False g (b <> TWhile g b t) t
```

Listing 3: Simplified version of the code used to unroll loops in the verifier.

3.1.2 Exceptions. During the building of the computation tree, we also do preparatory work for the exception throwing and handling logic. In our subset of GCL, we can statically determine which statements and expression can throw an exception. Additionally, we can also statically keep track of which try / catch handler is active at what part of the tree. These observations allow us to adjust the computation tree with extra branching statements to catch invalid behaviour and invoke the corresponding catch handler, when this occurs at run-time.

In particular, in the build function, we added an extra case that builds the tree for the try and catch blocks like regular. However, we add a post-processing step that inspects the statements and expression therein for potential exception throws.

- An array assignment, $a[i] = e$; s is replaced with a branch statement if $i \geq 0 \ \&\& \ i < \#a$ then $\{ s \}$ else $\{ e := 1 ; \text{handler} \}$
- Within other statements, we check each subexpression – when the statement has them – for their *throwing condition*. The throwing condition of an expression e is a predicate p such that assuming p will lead to an exception in e . Expressions can have multiple throwing conditions, but we order them by depth in the expression tree, i.e. the exception that would occur “first” will be thrown first.

¹<https://github.com/stickyPiston/infopv>

²<https://github.com/wooshrow/gclparser>

The throwing conditions for an expression are converted into nested branches, one for each condition. In the case none of the conditions are met, then the statement is executed as regular. In case of an exception, we assign the corresponding error code to the error variable and append the handler computation tree we computed earlier.

We chose for this approach because of its relatively simple implementation, but also because of its commensalism with the pruning system in the verifier. The pruning system finds infeasible branches and removes them from the computation tree. This means that, if an exception can only or cannot happen because of assumptions made earlier, then the branch is collapsed into single path, and the exception loses any overhead.

3.2 Verifier

We use forwards symbolic execution to run verification over our computation tree. We define the verifier as a function that takes a symbolic state and a computation tree and returns whether the program is valid. In the implementation we use a number of monads to help with keeping track of the symbolic state, updating statistics about the verifier’s execution and randomness. The symbolic state is described in Listing 4.

```
data SymbolicState = SymbolicState
{ environment      :: M.Map String (Expr Typed)
, equalities       :: M.Map String (Expr Typed)
, constraints       :: [Expr Typed]
, pathLength       :: Int
-- Constant parameters
, maxLength        :: Int
, pruneHeuristic   :: PruneHeuristic
, simplifyEnabled   :: Bool
}
```

Listing 4: Definition of the symbolic state as used in the implementation of the forwards symbolic verifier.

Symbolic execution recurses down the computation tree while continuously updating the symbolic state.

- In case of a branch, we recurse down into each subtree. However, in the then-branch we add the assumption that the guard holds, and in the else branch we assume the negation of the guard.
- In case of an assignment, we update the environment in the symbolic state for all of the subtrees. Specifically, we overwrite the value with the variable name as key with the evaluated right-hand expression. Evaluation of expressions means that all of the free variables are substituted for their values in the symbolic state’s environment. This assures that any value that is used in the SAT solver or that is saved in the environment uses the same “version” of the variables. That is, all propositions to the SAT solver are expressed from the symbolic initial values for each variable.
- In case of an assumption, we evaluate the predicate in the statement and add it to the list of assumption for the subtree.

- In case of an assertion, we again evaluate the predicate, but then use a SAT solver to check validity of the predicate. Specifically, let A denote the conjunction of the assumptions and p the predicate of the assertion, then if $\neg(A \implies p)$ is unsatisfiable, we continue the verification of the subtree, else we found an invalid path and stop the verification and report the violated assertion to the user.

3.2.1 Execution statistics. In order to evaluate the performance of the verifier, we also added statistics that are kept track of during the symbolic execution. In particular, we keep track of

- The number of full paths, i.e. the number of times the symbolic execution hits an `Empty`.
- The number of times the verifier could prune a branch. Note that this statistic does not tell how many branches were pruned, since an infeasible branch could contain many subtrees that we do not count.
- The number of paths that were longer than the given program length parameter.
- The total size of all formulae checked by the SAT solver.
- The number of times the SAT solver has been invoked.

3.3 Simplifier

We have implemented a simplifier to reduce the total complexity of the formulae sent to the SAT solver. Additionally, whenever the simplifier is able to reduce an expression down to trivial boolean expression, the SAT solver need not be invoked at all. This helps in reducing the time spent in the SAT solver, making checking a path less expensive to do. We used the following techniques in our simplifier:

3.3.1 Equalities as substitutions. The simplifier looks for assumptions in the form of $x=e$ where x is a variable name and e is a concrete variable. In the symbolic state, there is an additional environment called `equalities` in which such equalities are saved. During evaluation of expressions, the equalities are used as substitutions in a separate pass as the substitutions from the regular environment.

There are some restrictions to which equalities are used, which we will discuss below:

- We only create substitutions for *concrete variables*. A concrete variable is a variable whose value is known at verify-time, unlike symbolic variables. That is, the evaluated version of e reduces to a boolean or integer literal.
- We only look for equalities in assumptions at the root of the expression tree. That means that only statements of the form `assume x=e` will be detected. If the equality is in the branch of a condition or implication, then the conditions would need to be applied to the value on the right-hand side. While this is certainly possible to implement, we did not think it would be worth the effort, given that the expressions become larger, and the simplifier as-is might not be capable of simplifying them to a useful form.

3.3.2 Constant folding. Constant folding is a simplification that tries to combine constants as much as possible. For example, any operator applied on two concrete values can be reduced to the result of that computation. One particular reduction to highlight is the

disjunctive/conjunctive chain reduction, which collapses an entire chain of disjunctions when a trivially true value is found, or a chain of conjunctions when a false value is found.

3.3.3 Repby spine reduction. Lastly, we try to reduce the number of repby expressions in lists. In an array indexing expression with a concrete index, we check whether the requested index appears in the repby spine – the nested repby expressions. If that is the case, then we can substitute the array indexing expression with the expression at the found index. Similarly, if the index cannot be found in the repby spine, we remove all the repbys in the array index.

3.4 Pruning

As mentioned before, a common phenomenon in program verifiers is path explosion: at every branch, the number of paths doubles. Therefore, it is worth the effort to reduce the total number of paths as much as possible. One common technique is pruning, which looks at paths that have contradicting assumptions, and hence cannot be triggered. The path is then called infeasible. Pruning based on infeasibility does not compromise completeness.

3.4.1 Heuristic. To analyse the performance impact of the pruning, we implemented a simple heuristic that determines when to prune. When checking a branch for an infeasible path, the SAT solver has to be called, which can be quite expensive to do, performance wise. To try and create a balance between pruning enough to reduce the amount of inspected paths, and preventing calls to the solver when the payout of pruning would be low, we prune based on the path length. We generate a random number based on the current length of the path and compare that to the maximum path length provided at the start. This way, we try to prune a lot at the start of the program, when paths still have the potential to be very long, and decrease the frequency along the way.

4 Benchmarking results

The previously mentioned GCL parser library we used also included multiple benchmark GCL programs. We used these to analyse the performance of our verifier. The programs all contain an experiment parameter N , used to give concrete values to array lengths and loop conditions, thus influencing the complexity of the program. Each program also comes in two variants, one with a valid and one with an invalid postcondition. To run and generate reports of the benchmarks, the `criterion`³ library was used. The full reports of the benchmark can also be found in the repository in the `bench/reports` directory. All benchmarks were run on a system with a AMD Ryzen 7 7840HS CPU and 32 GB of RAM.

4.1 Verifier completeness

To analyse the completeness of our verifier, we run it on the provided invalid variants of the benchmark programs and measure the time needed to find the violation against an increasing N , with $N \in [2..10]$, with a fixed maximum path length K for each program. In all programs, the invalid postcondition could be correctly identified. Results of these benchmarks can be found in figure 1 through figure 4.

³<https://hackage.haskell.org/package/criterion>

4.2 Heuristic performance

To analyse the performance impact of our two heuristics, the simplifier and the pruning, we run the valid variants of the benchmark programs, with a fixed $N = 10$ and vary the maximum path length K . The simplifier can either be fully turned on or turned off. The pruning can be set to never try to prune, prune based on the length as mentioned in 3.4.1, or always try. Combining all options of the heuristics gives us six different options to benchmark per program. Results of these benchmarks can be found in figure 5 through figure 8.

Because the `find12` program exploits how errors are handled, our verifier will throw an error. There is a variable r that is never assigned to in the exception handler, but is used later in the program. However, because there is no way for an exception to be thrown in the valid variant of the program, the faulty but infeasible branches can be pruned. The `find12` program was therefore only tested with the full pruning enabled, to ensure only valid runs of the program were tested.

Performance analysis of our simplification and pruning heuristics shows that both optimizations significantly influence verification efficiency. The simplifier effectively reduces solver workload by collapsing symbolic expressions into simpler forms, particularly visible in array-heavy benchmarks such as `pullUp`. The pruning heuristic, successfully eliminates infeasible execution paths early, mitigating the path explosion problem inherent to symbolic execution. However, overly aggressive pruning can sometimes incur solver overhead when the payout is low, suggesting a balance between pruning frequency and program depth is important.

Across all benchmarks, the heuristics improved runtime scalability while preserving correctness. The experiments with varying program depths reveal that simplification provides consistent benefits across configurations, whereas pruning’s effectiveness depends on the branching structure of the program.

4.3 Program statistics

Apart from benchmarking on time, we have also evaluated the performance of our verifier based on statistics on the number of inspected paths, pruned paths, total formula size and invocations to the SAT solver. Below we will discuss the performance per program we have benchmarked on. Generally, we enabled the simplifier and try to prune at every branch, to test whether our optimisations are effective at verifying programs more efficiently.

4.3.1 `divByN`. The `divByN` program checks whether any x is divisible by N . This program contains a nested while loop, where the inner while simply increments a number until it equals N . Because the values in the inner loop are all concrete, our verifier can execute the while loop without the need for a SAT solver. This is reflected in the statistics in Table 1. We see that the total formula size sent to the SAT solver is quite small.

Additionally, we see that the path too long is always 1, because the verifier has to continue verifying until the path length runs out. And as the N increases, the number of full paths decreases. And, by extension, the number of SAT invocations and formula size decrease too. The reason is that the inner loop takes up more of the program length as N increases.

N	full	pruned	too long	for. size	z3 invoc.
2	16	50	1	374	50
3	13	54	1	331	41
4	11	57	1	303	35
5	9	60	1	275	29
6	8	62	1	262	26
7	7	63	1	247	23
8	6	64	1	232	20
9	6	65	1	234	20
10	5	66	1	219	17

Table 1: Statistics for the divByN program with full pruning, program length of 150 and the simplifier enabled.

4.3.2 memberOf. The memberOf programs checks whether an element a occurs in a list with length of at least N . This program can generate up to 2^N different paths. We see in Table 2 that the number of pruned paths increases with N . This is because the while cannot be exited before k is at least N . This gives the verifier more unfeasible paths to prune.

Having said that, our verifier can only handle this program for $N < 50$. After that, the process is killed for allocating too much memory.

N	full	pruned	too long	for. size	z3 invoc.
2	4082	13	4096	49114	20462
3	4079	16	4096	49108	20459
4	4072	23	4096	49094	20452
5	4057	38	4096	49064	20437
6	4026	69	4096	49002	20406
7	3963	132	4096	48876	20343
8	3836	259	4096	48622	20216
9	3581	514	4096	48112	19961
10	3070	1025	4096	47090	19450

Table 2: Statistics for the memberOf program with full pruning, program length of 50 and the simplifier enabled.

4.3.3 pullUp. The pullUp program tests how the verifier copes with array assignments. With naive symbolic execution, the program can generate an exponential number of repby expressions. However, as we can see in Table 3, the formula size grows proportionally to the number of paths. This indicates that the simplifier has been successful in reducing the number of repby expressions that is passed to the SAT solver.

4.3.4 find12. The find12 program is a test program for exception handling. The program has a precondition such that no exceptions will be thrown. And our verifier makes use of this fact: the number of inspected paths stays 1 regardless of the input list’s length. The statistics are summarised in Table 4. Additionally, we see that the number of pruned paths increases linearly with N . This is also expected since another precondition in the program forces the verifier to explore every iteration of the while loop.

N	full	pruned	too long	for. size	z3 invoc.
2	2	2	0	26	4
3	4	6	0	66	10
4	8	14	0	148	22
5	16	30	0	314	46
6	32	62	0	648	94
7	64	126	0	1318	190
8	128	254	0	2660	382
9	256	510	0	5346	766
10	512	1022	0	10720	1534

Table 3: Statistics for the pullUp program with full pruning, program length of 150 and the simplifier enabled.

N	full	pruned	too long	for. size	z3 invoc.
2	1	12	0	54	6
3	1	17	0	82	10
4	1	22	0	110	14
5	1	27	0	138	18
6	1	32	0	166	22
7	1	37	0	194	26
8	1	42	0	222	30
9	1	47	0	250	34
10	1	52	0	278	38

Table 4: Statistics for the find12 program with full pruning, program length of 150 and the simplifier enabled.

5 Conclusion

In this work, we presented BOOST, a bounded forward symbolic execution engine for the GCL language. BOOST successfully verified multiple benchmark programs featuring branching, loops, and exception handling. Our benchmark results demonstrate that BOOST achieves completeness across all invalid benchmark variants, correctly identifying violations within bounded depths, as well as concluding that a program is valid in reasonable time using our simplifier and pruning techniques. Together, these results validate the design of BOOST as an effective and extensible forward symbolic verifier that balances soundness, performance, and practicality for bounded program verification.

References

- [1] BALDONI, R., COPPA, E., D’ELIA, D. C., DEMETRESCU, C., AND FINOCCHI, I. A survey of symbolic execution techniques. *ACM Comput. Surv.* 51, 3 (2018).
- [2] CADAR, C., DUNBAR, D., AND ENGLER, D. Klee: unassisted and automatic generation of high-coverage tests for complex systems programs. In *Proceedings of the 8th USENIX Conference on Operating Systems Design and Implementation (USA, 2008)*, OSDI’08, USENIX Association, p. 209–224.
- [3] CADAR, C., AND SEN, K. Symbolic execution for software testing: three decades later. *Commun. ACM* 56, 2 (Feb. 2013), 82–90.
- [4] KAPUS, T., BUSSE, F., AND CADAR, C. Pending constraints in symbolic execution for better exploration and seeding. In *2020 35th IEEE/ACM International Conference on Automated Software Engineering (ASE) (2020)*, pp. 115–126.
- [5] KUZNETSOV, V., KINDER, J., BUCUR, S., AND CANDEA, G. Efficient state merging in symbolic execution. *SIGPLAN Not.* 47, 6 (June 2012), 193–204.
- [6] SCHWARTZ, E. J., AVGERINOS, T., AND BRUMLEY, D. All you ever wanted to know about dynamic taint analysis and forward symbolic execution (but might have been afraid to ask). In *2010 IEEE Symposium on Security and Privacy (2010)*, pp. 317–331.
- [7] SHUAI, Z., CHEN, Z., MA, K., LIU, K., ZHANG, Y., SUN, J., AND WANG, J. Partial

solution based constraint solving cache in symbolic execution. *Proc. ACM Softw. Eng.* 1, FSE (July 2024).

- [8] TRABISH, D., RINETZKY, N., SHOHAM, S., AND SHARMA, V. State merging with quantifiers in symbolic execution, 2023.

A Appendix: Bechmarking graphs

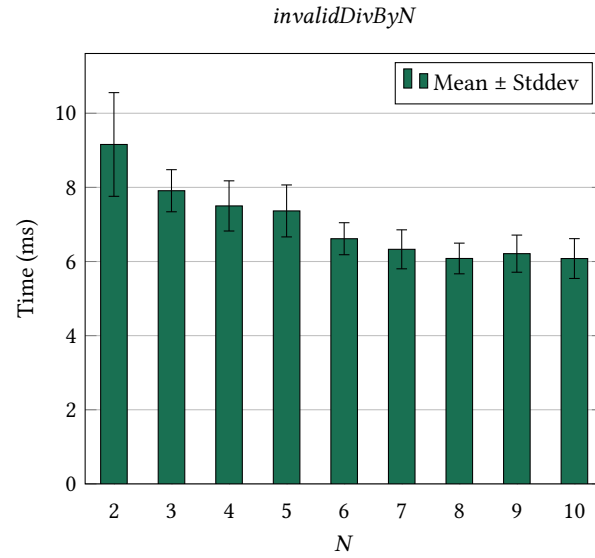


Figure 1: Completeness benchmark for *invalidDivByN* with $K = 100$, pruning disabled and the simplifier enabled.

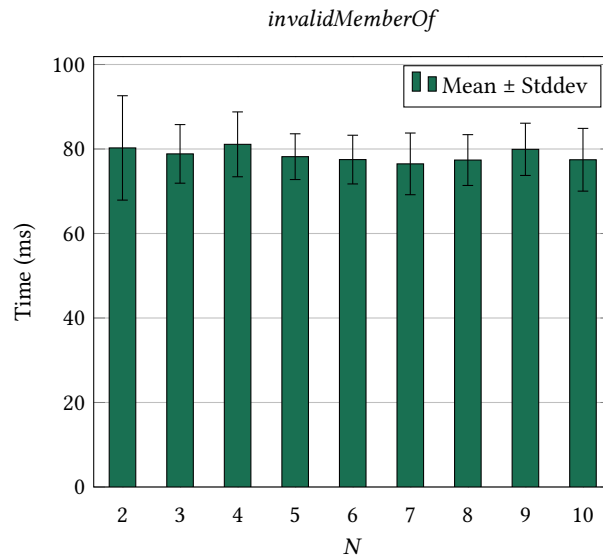


Figure 2: Completeness benchmark for *invalidMemberOf* with $K = 50$, pruning disabled and the simplifier enabled.

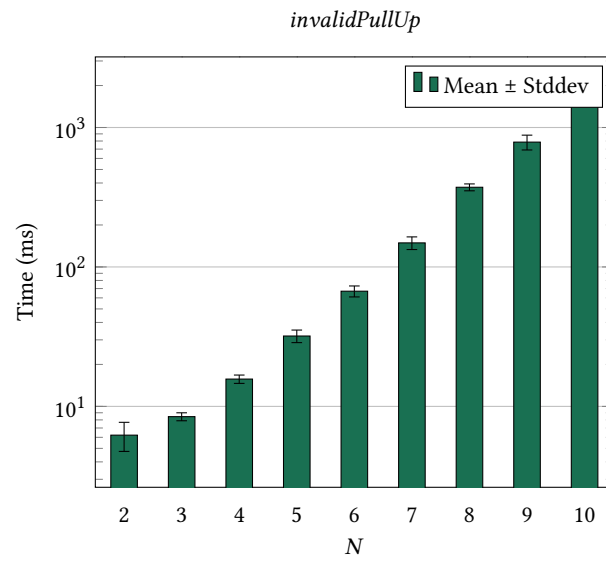


Figure 3: Completeness benchmark for *invalidPullUp* with $K = 50$, full pruning, and the simplifier enabled.

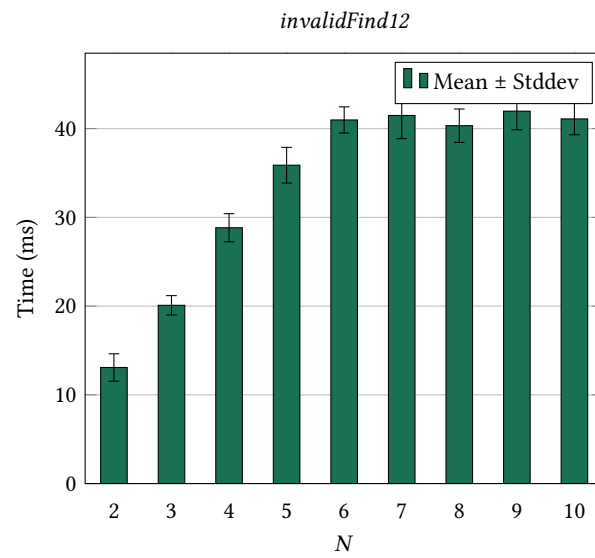


Figure 4: Completeness benchmark for *invalidFind12* with $K = 50$, full pruning and the simplifier enabled.

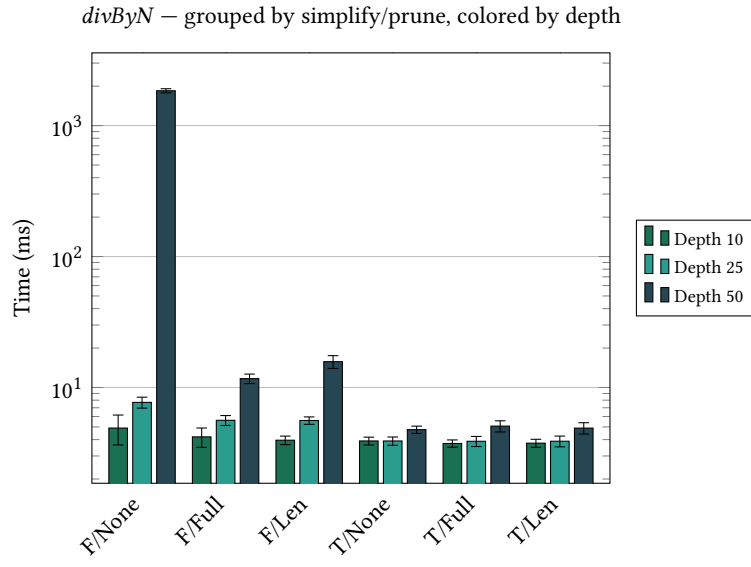


Figure 5: Heuristic benchmark of *divByN* with $N = 10$ across simplify/prune settings, for depths 10, 25, and 50.

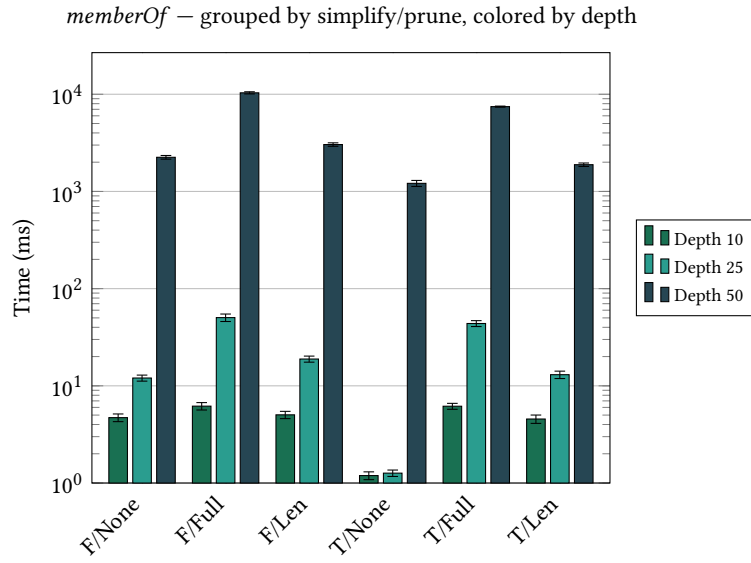


Figure 6: Heuristic benchmark of *memberOf* with $N = 10$ across simplify/prune settings, for depths 10, 25, and 50.

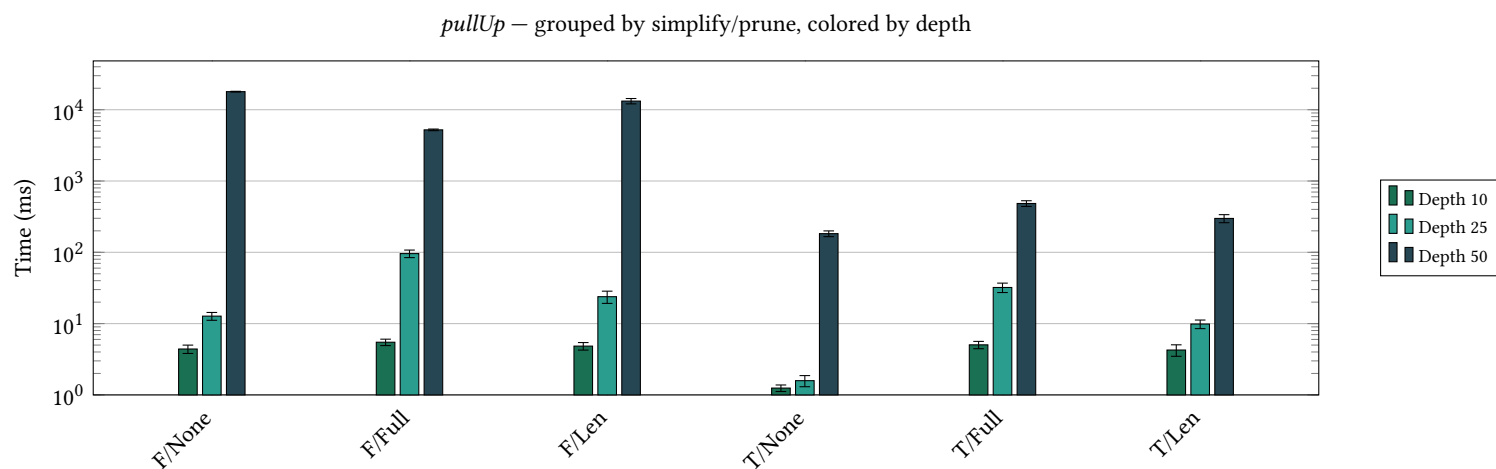


Figure 7: Heuristic benchmark of *pullUp* with $N = 10$ across simplify/prune settings, for depths 10, 25, and 50.

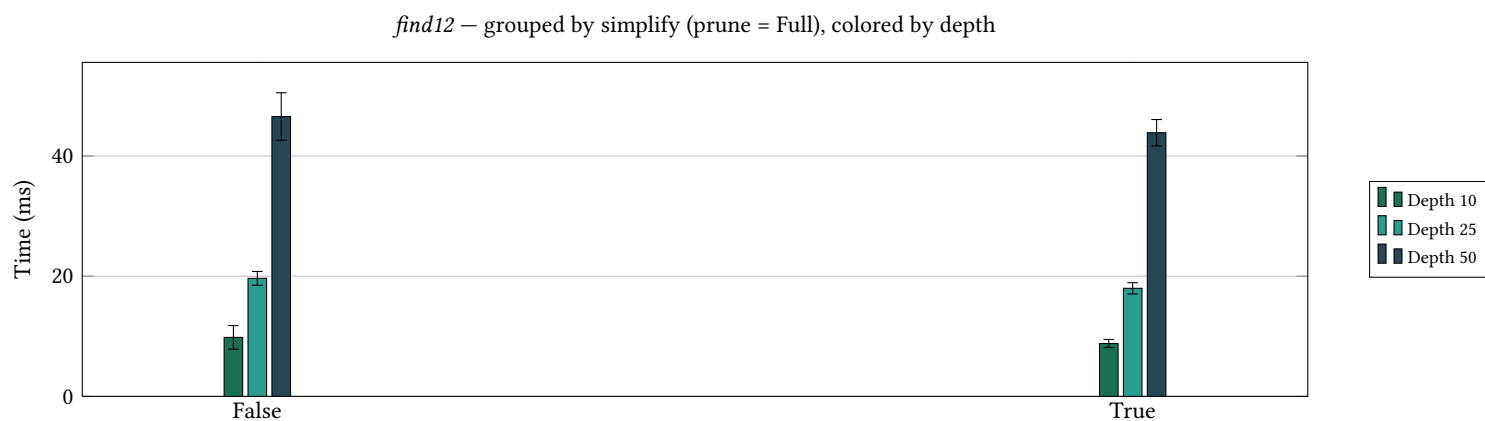


Figure 8: Heuristic benchmark of *find12* with $N = 10$, full pruning, across simplify setting, for depths 10, 25, and 50.